

TRABALHO PRÁTICO – PARTE 5

Criptografia XOR e Algoritmos de Casamento de Padrões

Disciplina: Algoritmos e Estruturas de Dados III

Pontifícia Universidade Católica de Minas Gerais

Alunos:

Samuel Correia

Matheus Augusto

2026

1. Introdução

A Parte 5 do Trabalho Prático consiste na implementação de criptografia sobre o sistema de gerenciamento de personagens World of RPGcraft, desenvolvido em Python. O objetivo é proteger as senhas dos usuários armazenadas nos arquivos binários do banco de dados, aplicando o algoritmo de cifra XOR (ou-exclusivo), além de implementar os algoritmos de busca (KMP e Boyer–Moore.)

2. Conceito da Criptografia XOR

O operador XOR (ou-exclusivo bit a bit) possui uma propriedade fundamental que o torna ideal para criptografia simétrica:

```
A XOR B = C      # cifrar
C XOR B = A      # decifrar (mesma operação!)
```

Isso significa que a mesma função serve tanto para cifrar quanto para decifrar — basta aplicar novamente com a mesma chave. A tabela verdade do XOR é:

Bit A	Bit B	A XOR B	Resultado
0	0	0	Iguals → 0
0	1	1	Diferentes → 1
1	0	1	Diferentes → 1
1	1	0	Iguals → 0

Propriedade-chave: a operação XOR é simétrica e reversível. Não há distinção entre "cifrar" e "decifrar" — o algoritmo é idêntico nos dois sentidos.

3. Arquivos Criados e Modificados

Arquivo	Status	Descrição
criptografia.py	NOVO	Módulo com as funções XOR: xor_cifrar(), xor_decifrar(), xor_verificar()
controller/CasamentoPadroes.py	NOVO	Algoritmos de casamento de padrões: kmp_buscar() e boyer_moore_buscar()
model/Conta.py	MODIFICADO	Adicionado campo senha_cifrada (20 bytes) e métodos set_senha() / verificar_senha()
dao/PersonagemDAO.py	MODIFICADO	Adicionado buscar_por_padrao_nome() para busca textual via KMP/BM
server.py	MODIFICADO	Login com senha + busca por padrão na página de personagens

main.py	MODIFICADO	Menu CLI com opção de pesquisa por padrão (KMP / BM)
controller/HashExtensivel.py	CORRIGIDO	Hash de strings trocado de hash() para MD5 determinístico (hashlib)

4. Módulo criptografia.py

O módulo centraliza toda a lógica de criptografia XOR. Nenhum outro módulo do sistema precisa conhecer os detalhes da cifra — basta importar as três funções públicas.

4.1 Chave da Cifra

Chave utilizada: "WoWRPGcraft2026!" (16 bytes). A chave é aplicada ciclicamente sobre os bytes da senha — se a senha for mais longa que 16 caracteres, a chave recomeça do byte inicial.

5. Integração com o Modelo Conta

5.1 Novo formato do registro

O campo senha_cifrada foi adicionado ao final do registro binário de Conta. O tamanho do registro aumentou de 65 para 85 bytes:

```
# ANTES (65 bytes)
FORMATO = "<1s i 20s 30s 10s"
# Lápide(1) + ID(4) + Usuário(20) + Email(30) + Data(10)

# DEPOIS (85 bytes)
FORMATO = "<1s i 20s 30s 10s 20s"
# Lápide(1) + ID(4) + Usuário(20) + Email(30) + Data(10) + Senha_XOR(20)
```

5.2 Métodos adicionados à classe Conta

```
def set_senha(self, senha_texto: str):
    self.senha_cifrada = xor_cifrar(senha_texto[:20])

def verificar_senha(self, senha_texto: str) -> bool:
    return xor_verificar(senha_texto, self.senha_cifrada)
```

O modelo nunca armazena a senha em texto claro — apenas os bytes resultantes da operação XOR. O método verificar_senha() cifra a entrada e compara os bytes sem precisar decifrar o valor armazenado.

6. Perguntas e respostas do relatório

Esta seção responde às questões teóricas e práticas sobre a implementação dos algoritmos de casamento de padrões (KMP e Boyer-Moore) e da criptografia.

1. Qual campo textual foi escolhido para aplicar os algoritmos de casamento de padrões? Por quê?

Foi escolhido o campo **nome** da tabela **Personagem**.

Motivos:

- É o campo textual de busca mais natural para o usuário — faz sentido procurar um herói por parte do nome (ex.: "art" encontra "Arthas").
- Tem conteúdo livre digitado pelo usuário, ao contrário de campos como **funcao** (que só aceita dano/tanque/suporte) ou campos numéricos (id, nível).
- A busca é **case-insensitive**: tanto o nome quanto o padrão são convertidos para minúsculas antes da comparação.

2. Explique o funcionamento do KMP implementado

O KMP (Knuth-Morris-Pratt) evita retroceder no texto reaproveitando o que já foi comparado. Funciona em duas etapas:

Etapa 1 - Tabela de falha (_kmp_tabela_falha): para cada posição do padrão, calcula o comprimento do maior prefixo que também é sufixo daquele trecho. Essa tabela indica para onde voltar no padrão quando ocorre uma incompatibilidade, sem precisar voltar no texto.

```
padrão "abab" -> tabela [0, 0, 1, 2]
```

Etapa 2 - Busca (kmp_buscar): percorre o texto uma única vez (o índice *i* nunca retrocede), mantendo um contador *k* de quantos caracteres do padrão já casaram.

- Em caso de incompatibilidade, em vez de recomeçar, consulta a tabela: $k = \text{tabela}[k-1]$.
- Quando *k* atinge o tamanho do padrão, registra a ocorrência e continua usando a tabela para achar ocorrências sobrepostas.

Complexidade: $O(n + m)$, onde *n* = tamanho do texto e *m* = tamanho do padrão. Retorna a lista de todas as posições onde o padrão aparece.

3. Explique o funcionamento do Boyer-Moore implementado

O Boyer-Moore compara o padrão da direita para a esquerda e, ao falhar, pula vários caracteres de uma vez. Foram implementadas as duas heurísticas:

Heurística do Bad Character : monta um dicionário com a última posição de cada caractere no padrão. Quando um caractere do texto não casa, o padrão é deslocado para alinhar esse caractere com sua última ocorrência no padrão (ou pula o padrão inteiro se o caractere não existir nele).

Heurística do Good Suffix: quando um sufixo do padrão já casou antes de uma falha, desloca o padrão para reaproveitar esse sufixo, procurando outra ocorrência dele ou um prefixo que coincida com ele.

Busca : em cada alinhamento, compara da direita para a esquerda. Em caso de falha, aplica o maior dos dois deslocamentos: $\max(1, \text{desloc_bad}, \text{desloc_good})$, garantindo pulos maiores e nunca menos de 1 posição.

Vantagem: em textos comuns, costuma examinar menos caracteres que o tamanho do texto, sendo geralmente mais rápido que o KMP na prática.

Validação: foi verificado que KMP e Boyer-Moore retornam exatamente as mesmas posições em todos os casos de teste (padrões repetidos, vazios e múltiplas ocorrências).

4. Como integrou os algoritmos ao sistema.

A integração foi feita em três camadas:

- **Camada de algoritmo (controller/CasamentoPadroes.py)**: funções puras `kmp_buscar` e `boyer_moore_buscar`, independentes do resto do sistema.
- **Camada de dados (dao/PersonagemDAO.py)**: método `buscar_por_padrao_nome(padrao, algoritmo)` que varre sequencialmente o arquivo binário `personagens.bin`, ignora registros excluídos (lápide) e aplica o algoritmo escolhido sobre o nome de cada registro ativo.
- **Camada de interface (CLI e Web)**: no CLI (`main.py`), opção 5 do menu "Pesquisar por padrão (KMP / BM)"; na Web (`server.py` + `templates/personagens.html`), um card de busca na página de personagens com campo de texto, seletor de algoritmo e tabela de resultados.

5. Quais dificuldades encontrou na implementação dos dois algoritmos?

- **Tabela de Good Suffix do Boyer-Moore**: é a parte mais complexa — envolve o conceito de bordas e duas varreduras do padrão; errar índices gera resultados sutilmente incorretos.
- **Garantir deslocamento mínimo de 1**: sem o $\max(1, \dots)$, certas falhas poderiam travar o Boyer-Moore em loop infinito.
- **Ocorrências sobrepostas no KMP**: ao encontrar uma ocorrência, era preciso fazer $k = \text{tabela}[k-1]$ (e não zerar k) para não perder casamentos que se sobrepõem (ex.: "aa" em "aaaa").
- **Casos de borda**: padrão vazio, padrão maior que o texto e texto vazio — tratados explicitamente no início de cada função.
- **Validação cruzada**: garantir que os dois algoritmos produzissem resultados idênticos exigiu um conjunto de testes comparativos.

6. Qual campo foi utilizado na criptografia?

O campo **senha** (`senha_cifrada`) da tabela **Conta** — um campo de 20 bytes adicionado ao registro especificamente para a Parte 5. É o campo sensível por excelência, pois protege o acesso à conta do usuário.

7. Qual foi o método utilizado na criptografia?

A cifra **XOR (ou-exclusivo)**, implementada em `criptografia.py`.

- **Chave**: "WoWRPGcraft2026!" (16 bytes), aplicada ciclicamente sobre os bytes da senha.

- **Propriedade central:** o XOR é simétrico — a mesma operação cifra e decifra ($A \text{ XOR } B \text{ XOR } B = A$).
- **No login:** a senha digitada é cifrada e comparada byte a byte com a armazenada (xor_verificar), sem nunca guardar ou exibir a senha em texto claro.

Repositório GitHub

O código-fonte completo do projeto está disponível em:

<https://github.com/samulkil/AEDS3PersonagemdeWoW>